Interactive Theorem Proving with Isabelle/HOL

Mnacho Echenim, Nicolas Peltier

With contributions from Hervé Guiol, Mehdi Mhalla and several Ensimag students

Grenoble INP - UGA, LIG-CAPP

About this talk

- A quick introduction to interactive theorem proving
- Some transversal formalization efforts
  - **The Cox-Ross-Rubinstein model**
  - The CHSH inequality and Tsirelson's upper-bound
  - The Superposition Calculus
- Current formalization work on AMMs

About me

- Main research field: automated reasoning
  - A part of Symbolic AI
  - Deductive, inductive and abductive reasoning

About me

- Main research field: automated reasoning
  - A part of Symbolic AI
  - Deductive, inductive and abductive reasoning
- Identify new automated reasoning techniques
  - Automatically generating missing hypotheses
  - Reasoning on complex structures (heaps, graphs...)

About me

- Main research field: automated reasoning
  - A part of Symbolic AI
  - Deductive, inductive and abductive reasoning
- Identify new automated reasoning techniques
  - Automatically generating missing hypotheses
  - Reasoning on complex structures (heaps, graphs...)
- Use automated reasoning tools
  - In the context of Interactive Theorem Proving
  - Explore different formalization tasks

Interactive theorem proving

On formal proofs

- A formal proof is constructed from
  - A fixed set of axioms
  - A (limited) set of inference rules

On formal proofs

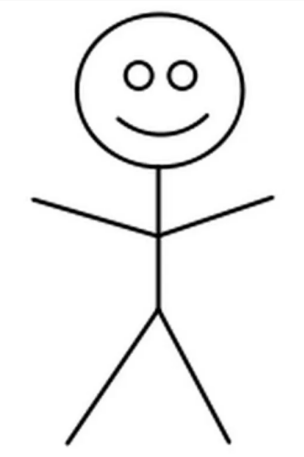
- A formal proof is constructed from
  - A fixed set of axioms
  - A (limited) set of inference rules
- Why they are important
  - They can be independently verified by any proof checker
  - There can be no gaps or mistakes in a formal proof
  - G. Gonthier: *"all you have to do is agree with the definitions"*

On formal proofs

- A formal proof is constructed from
  - A fixed set of axioms
  - A (limited) set of inference rules
- Why they are important
  - They can be independently verified by any proof checker
  - There can be no gaps or mistakes in a formal proof
  - G. Gonthier: *"all you have to do is agree with the definitions"*
- A formal proof can be very (very, very) long
  - They cannot reasonably be done by hand
  - Theoretical limitations to using automated theorem provers to construct every possible formal proof
  - The trade-off: **interactive theorem provers**

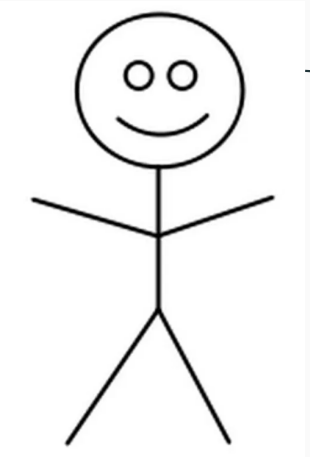
About interactive theorem proving

Principle: collaboration between a human and a program to construct formal proofs



About interactive theorem proving

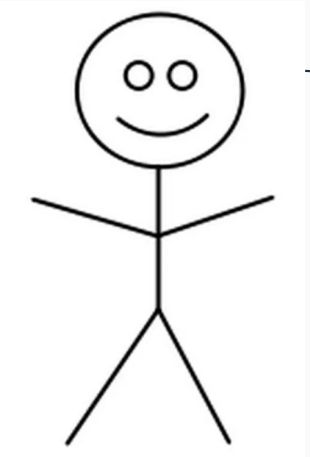
Principle: collaboration between a human and a program to construct formal proofs



Theorem

About interactive theorem proving

Principle: collaboration between a human and a program to construct formal proofs

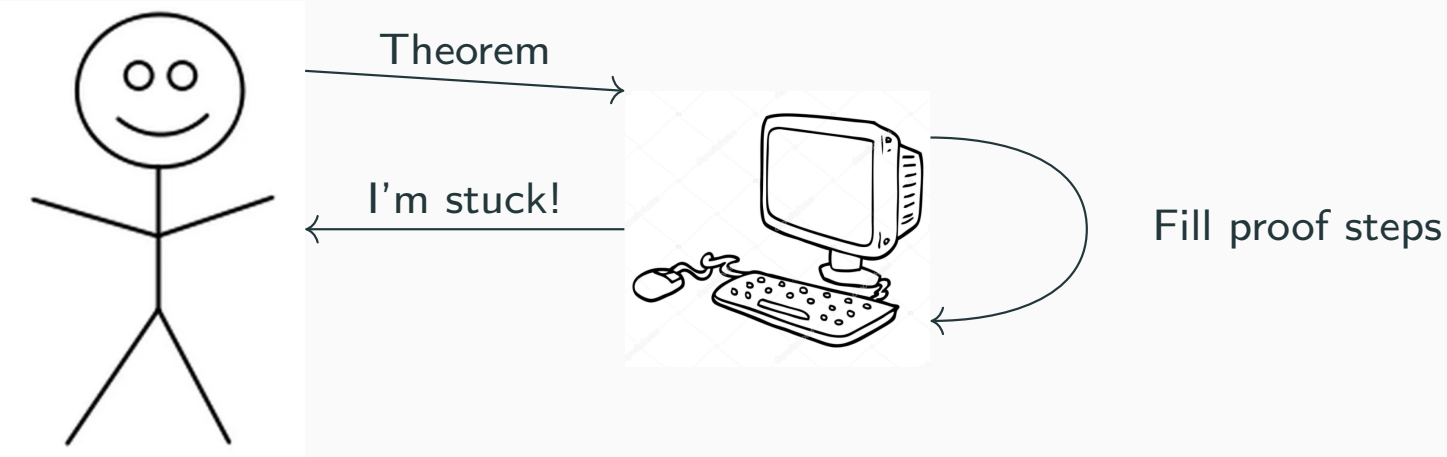


Theorem

Fill proof steps

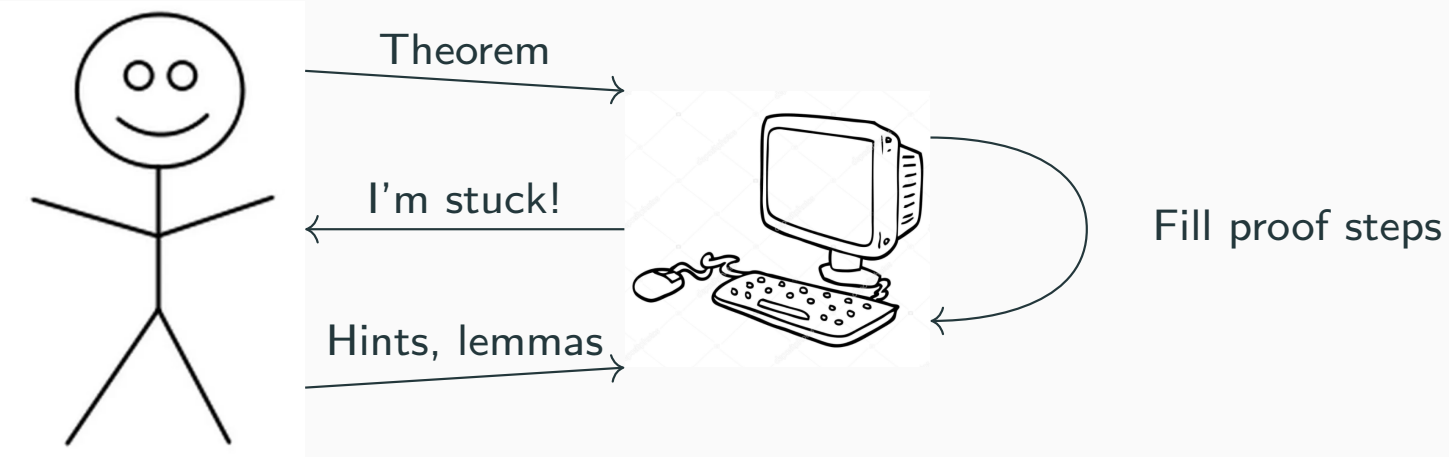
About interactive theorem proving

Principle: collaboration between a human and a program to construct formal proofs



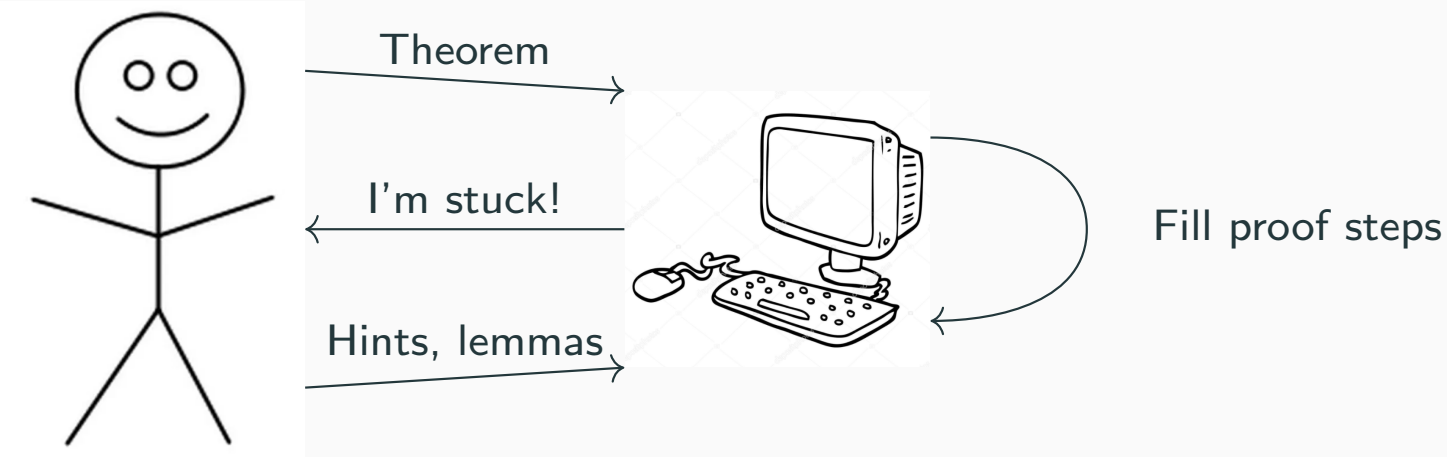
About interactive theorem proving

Principle: collaboration between a human and a program to construct formal proofs



About interactive theorem proving

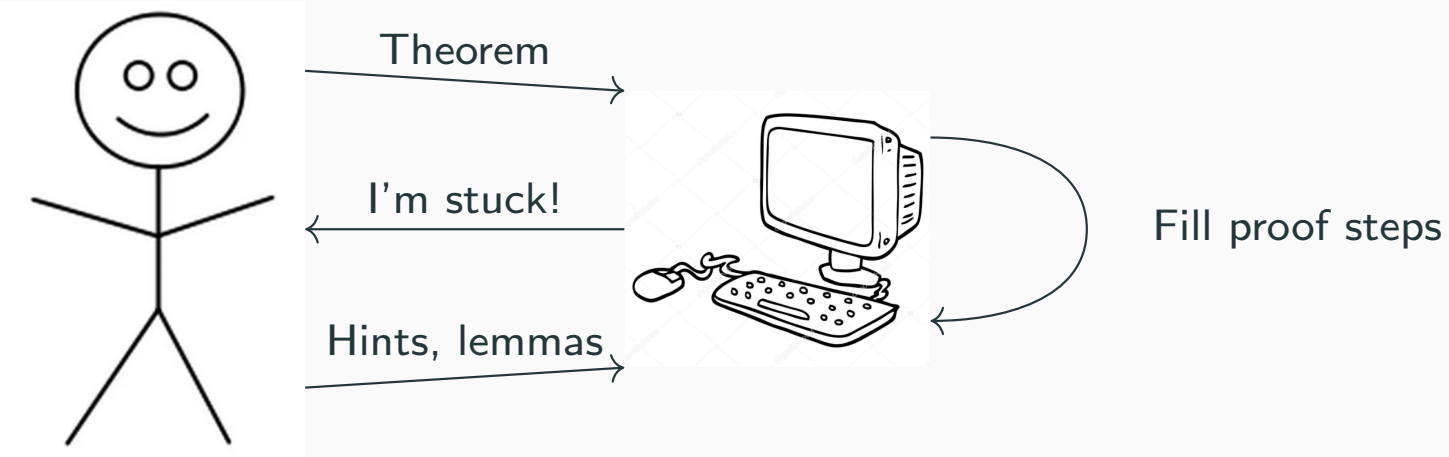
Principle: collaboration between a human and a program to construct formal proofs



- Other features
  - Theory exploration: *what happens when I change/relax a hypothesis?*
  - Extraction of source code that is correct by design

About interactive theorem proving

Principle: collaboration between a human and a program to construct formal proofs



- Other features
  - Theory exploration: *what happens when I change/relax a hypothesis?*
  - Extraction of source code that is correct by design
- Popular systems
  - Coq, **Isabelle**, Lean, ...

Some success stories

- In mathematics
  - Kepler's conjecture ([Hales], Flyspeck project)
  - Four-color theorem ([Gonthier, Werner])
  - Odd-order theorem ([Gonthier et al])

Some success stories

- In mathematics
  - Kepler's conjecture ([Hales], Flyspeck project)
  - Four-color theorem ([Gonthier, Werner])
  - Odd-order theorem ([Gonthier et al])
- In computer science
  - CompCert compiler (used in the Airbus toolchain)
  - sel4 microkernel, used in aviation, spaceflight
  - OpenSSL implementation of Keyed-Hash Message Authentication Code (HMAC)

Correcting mistakes

- Flyspeck project
  - Gap in the original proof of the Kepler conjecture, the authors wrote a few pages of human explanation about it
  - `https://www.cl.cam.ac.uk/~jrh13/papers/revkepler.pdf`

Correcting mistakes

- Flyspeck project
  - Gap in the original proof of the Kepler conjecture, the authors wrote a few pages of human explanation about it
  - `https://www.cl.cam.ac.uk/~jrh13/papers/revkepler.pdf`
- Morse lemma in Gromov hyperbolic spaces
  - Gap in the proof published in the Journal of Functional Analysis
  - Corrected proof: `https://www.math.sciences.univ-nantes.fr/~gouezel/articles/morse_lemma.pdf`

Correcting mistakes

- Flyspeck project
  - Gap in the original proof of the Kepler conjecture, the authors wrote a few pages of human explanation about it
  - https://www.cl.cam.ac.uk/~jrh13/papers/revkepler.pdf
- Morse lemma in Gromov hyperbolic spaces
  - Gap in the proof published in the Journal of Functional Analysis
  - Corrected proof: https://www.math.sciences.univ-nantes.fr/~gouezel/articles/morse_lemma.pdf
- Random dictatorship theorem
  - Wrong proof published in the Journal of Economic Letters
  - Corrected proof: https://www.sciencedirect.com/science/article/pii/S0165176516302063

About Isabelle

- Available at `https://isabelle.in.tum.de/`
- Simple type system, close to that of functional programming languages
    - But features such as type classes and locales help formalize theorems in a natural way
    - Some very advanced mathematics have been formalized in Isabelle

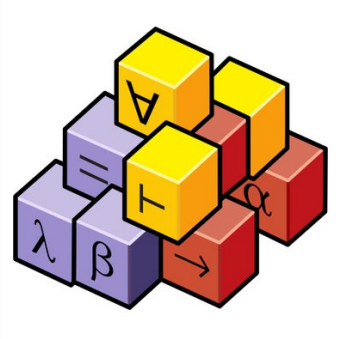
About Isabelle

- Available at `https://isabelle.in.tum.de/`
- Simple type system, close to that of functional programming languages
    - But features such as type classes and locales help formalize theorems in a natural way
    - Some very advanced mathematics have been formalized in Isabelle
- Large corpus of formalized notions
    - A new version of the tool every year
    - Archive of Formal Proofs: `https://www.isa-afp.org/`, refereed formalizations

About Isabelle

- Available at `https://isabelle.in.tum.de/`
- Simple type system, close to that of functional programming languages
  - But features such as type classes and locales help formalize theorems in a natural way
  - Some very advanced mathematics have been formalized in Isabelle
- Large corpus of formalized notions
  - A new version of the tool every year
  - Archive of Formal Proofs: `https://www.isa-afp.org/`, refereed formalizations
- Built-in tools for proving statements automatically
  - `simp`, `auto`, `blast`, `metis`, `sledgehammer`

About Isabelle



- Available at `https://isabelle.in.tum.de/`
- Simple type system, close to that of functional programming languages
  - But features such as type classes and locales help formalize theorems in a natural way
  - Some very advanced mathematics have been formalized in Isabelle
- Large corpus of formalized notions
  - A new version of the tool every year
  - Archive of Formal Proofs: `https://www.isa-afp.org/`, refereed formalizations
- Built-in tools for proving statements automatically
  - `simp`, `auto`, `blast`, `metis`, `sledgehammer`
- Built-in tools for disproving statements
  - Because many statements we write start out being false (missing hypothesis. . .)
  - `nitpick`, `quickcheck`

The Cox-Ross-Rubinstein model

About the formalization

- What was already available in Isabelle
    - Several fundamental notions of measure theory

About the formalization

- What was already available in Isabelle
  - Several fundamental notions of measure theory
- Goal of the formalization
  - Formalize missing notions from measure theory
    - Filtrations
    - Martingales
    - Adapted and predictable processes

About the formalization

- What was already available in Isabelle
  - Several fundamental notions of measure theory
- Goal of the formalization
  - Formalize missing notions from measure theory
    - Filtrations
    - Martingales
    - Adapted and predictable processes
  - Define basic financial notions
    - Portfolios, self-financing portfolios
    - Arbitrage processes and fair prices
    - Replicating portfolios and complete markets

About the formalization

- What was already available in Isabelle
  - Several fundamental notions of measure theory
- Goal of the formalization
  - Formalize missing notions from measure theory
    - Filtrations
    - Martingales
    - Adapted and predictable processes
  - Define basic financial notions
    - Portfolios, self-financing portfolios
    - Arbitrage processes and fair prices
    - Replicating portfolios and complete markets
  - Show market completeness under the Cox-Ross-Rubinstein hypothesis

About the formalization

- What was already available in Isabelle
  - Several fundamental notions of measure theory
- Goal of the formalization
  - Formalize missing notions from measure theory
    - Filtrations
    - Martingales
    - Adapted and predictable processes
  - Define basic financial notions
    - Portfolios, self-financing portfolios
    - Arbitrage processes and fair prices
    - Replicating portfolios and complete markets
  - Show market completeness under the Cox-Ross-Rubinstein hypothesis
  - Derive pricing functions for standard options that are correct by design

Discrete self-financing portfolios

Assumption: time is discrete, represented by $\mathbb{N}$.

Discrete self-financing portfolios

Assumption: time is discrete, represented by $\mathbb{N}$.

- A market consists of a set of assets, characterized by their price processes: `market-assets` $::$ $(\mathbb{N} \to \alpha \to \mathbb{R})$ `set`

Discrete self-financing portfolios

Assumption: time is discrete, represented by $\mathbb{N}$.

- A market consists of a set of assets, characterized by their price processes:
  `market-assets` :: $(\mathbb{N} \to \alpha \to \mathbb{R})$ `set`
- **typedef** α `disc-pf`
  A (discrete) portfolio p contains an evolving quantity of assets. Notations:
  - π_i^p represents the price process of the ith asset in p.
  - κ_i^p represents the quantity process of the ith asset in p.

Discrete self-financing portfolios

Assumption: time is discrete, represented by $\mathbb{N}$.

- A market consists of a set of assets, characterized by their price processes:
  `market-assets` $::$ $(\mathbb{N} \to \alpha \to \mathbb{R})$ `set`
- **typedef** α `disc-pf`
  A (discrete) portfolio p contains an evolving quantity of assets. Notations:
  - π_i^p represents the price process of the ith asset in p.
  - κ_i^p represents the quantity process of the ith asset in p.
- The value process of a portfolio p containing m assets is

$$\texttt{val-proc}\ p \ = \ \lambda n\ \omega.\ \sum_{i=1}^{m} (\pi_i^p\ n\ \omega).(\kappa_i^p\ n\ \omega)$$

Discrete self-financing portfolios

Assumption: time is discrete, represented by $\mathbb{N}$.

- A market consists of a set of assets, characterized by their price processes:
  `market-assets` $:: (\mathbb{N} \to \alpha \to \mathbb{R})$ `set`

- **typedef** α `disc-pf`
  A (discrete) portfolio p contains an evolving quantity of assets. Notations:

  - π_i^p represents the price process of the ith asset in p.
  - κ_i^p represents the quantity process of the ith asset in p.

- The value process of a portfolio p containing m assets is

$$\texttt{val-proc}\ p \ = \ \lambda n\ \omega.\ \sum_{i=1}^{m} (\pi_i^p\ n\ \omega).(\kappa_i^p\ n\ \omega)$$

- A portfolio with m assets is self-financing if $\forall n \geq 1$ and $\forall \omega \in \Omega$:

$$\texttt{val-proc}\ p\ (n+1)\ \omega \ = \ \sum_{i=1}^{m} (\pi_i^p\ (n+1)\ \omega) \cdot (\kappa_i^p\ n\ \omega)$$

Arbitrages, viable markets

$$
\begin{aligned}
&\texttt{arbitrage-process} && :: && (\alpha, \beta)\,\texttt{discrete-market} \to \alpha\ \texttt{disc-pf} \to \mathbb{B} \\
&\texttt{arbitrage-process}\ \texttt{Mkt}\ p && \Leftrightarrow && \exists N \in \mathbb{N}. \\
& && && (\texttt{trading-strat}\ p)\ \wedge\ (\texttt{self-financing}\ p)\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ 0\ =\ 0) = 1\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ N\ \geq 0) = 1\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ N\ > 0) > 0
\end{aligned}
$$

Arbitrages, viable markets

$$
\begin{aligned}
&\texttt{arbitrage-process} && :: && (\alpha, \beta)\,\texttt{discrete-market} \to \alpha\ \texttt{disc-pf} \to \mathbb{B} \\
&\texttt{arbitrage-process}\ \texttt{Mkt}\ p && \Leftrightarrow && \exists N \in \mathbb{N}. \\
& && && (\texttt{trading-strat}\ p)\ \wedge\ (\texttt{self-financing}\ p)\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ 0\ =\ 0) = 1\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ N\ \geq 0) = 1\ \wedge \\
& && && \mathbb{P}(\texttt{val-proc}\ p\ N\ > 0) > 0
\end{aligned}
$$

$$
\begin{aligned}
&\texttt{viable-market} && :: && (\alpha, \beta)\,\texttt{discrete-market} \to \mathbb{B} \\
&\texttt{viable-market}\ \texttt{Mkt} && \Leftrightarrow && \forall p.\ \texttt{stock-portfolio}\ p\ \Rightarrow \\
& && && \quad \neg\texttt{arbitrage-process}\ \texttt{Mkt}\ p
\end{aligned}
$$

Complete markets and fair prices

$$
\begin{aligned}
&\texttt{replic-pf} && :: && (\beta \to \mathbb{N} \to \alpha \to \mathbb{R}) \to (\alpha \to \mathbb{R}) \to \ \mathbb{N} \to \mathbb{B} \\
&\texttt{replic-pf}\ p\ \kappa\ T && \Leftrightarrow && (\texttt{stock-portfolio}\ \texttt{Mkt}\ \ p) \wedge \\
& && && (\texttt{self-financing}\ p) \wedge (\texttt{trading-strat}\ p) \wedge \\
& && && (\mathrm{AE}_{\mathcal{M}}\ w.\ \texttt{val-proc}\ \texttt{Mkt}\ p\ T\ w = \kappa\ w)
\end{aligned}
$$

Complete markets and fair prices

$$
\begin{aligned}
&\texttt{replic-pf} && :: && (\beta \to \mathbb{N} \to \alpha \to \mathbb{R}) \to (\alpha \to \mathbb{R}) \to \ \mathbb{N} \to \mathbb{B} \\
&\texttt{replic-pf}\ p\ \kappa\ T && \Leftrightarrow && (\texttt{stock-portfolio}\ \texttt{Mkt}\ \ p) \wedge \\
& && && (\texttt{self-financing}\ p) \wedge (\texttt{trading-strat}\ p) \wedge \\
& && && (\mathrm{AE}_{\mathcal{M}}\ w.\ \texttt{val-proc}\ \texttt{Mkt}\ p\ T\ w = \kappa\ w)
\end{aligned}
$$

$$
\begin{aligned}
&\texttt{attainable} && :: && (\alpha \to \mathbb{R}) \to \mathbb{N} \to \mathbb{B} \\
&\texttt{attainable}\ \kappa\ T && \Leftrightarrow && (\exists p.\ \texttt{replic-pf}\ p\ \kappa\ T)
\end{aligned}
$$

Complete markets and fair prices

$$
\begin{aligned}
&\texttt{replic-pf} && :: && (\beta \to \mathbb{N} \to \alpha \to \mathbb{R}) \to (\alpha \to \mathbb{R}) \to \ \mathbb{N} \to \mathbb{B} \\
&\texttt{replic-pf}\ p\ \kappa\ T && \Leftrightarrow && (\texttt{stock-portfolio}\ \texttt{Mkt}\ \ p) \wedge \\
& && && (\texttt{self-financing}\ p) \wedge (\texttt{trading-strat}\ p) \wedge \\
& && && (\mathrm{AE}_{\mathcal{M}}\ w.\ \texttt{val-proc}\ \texttt{Mkt}\ p\ T\ w = \kappa\ w)
\end{aligned}
$$

$$
\begin{aligned}
&\texttt{attainable} && :: && (\alpha \to \mathbb{R}) \to \mathbb{N} \to \mathbb{B} \\
&\texttt{attainable}\ \kappa\ T && \Leftrightarrow && (\exists p.\ \texttt{replic-pf}\ p\ \kappa\ T)
\end{aligned}
$$

$$
\begin{aligned}
&\texttt{complete-market} && :: && (\alpha, \beta)\,\texttt{discrete-market} \to \mathbb{B} \\
&\texttt{complete-market}\ \texttt{Mkt} && \Leftrightarrow && \forall T.\ \forall \kappa \in (\texttt{borel-measurable}\ \mathcal{F}_T).\ \texttt{attainable}\ \kappa\ T
\end{aligned}
$$

Complete markets and fair prices

$$\texttt{replic-pf} \quad :: \quad (\beta \to \mathbb{N} \to \alpha \to \mathbb{R}) \to (\alpha \to \mathbb{R}) \to \mathbb{N} \to \mathbb{B}$$

$$\begin{aligned}\texttt{replic-pf}\ p\ \kappa\ T \quad \Leftrightarrow \quad & (\texttt{stock-portfolio}\ \texttt{Mkt}\ \ p) \wedge \\ & (\texttt{self-financing}\ p) \wedge (\texttt{trading-strat}\ p) \wedge \\ & (\mathrm{AE}_{\mathcal{M}}\ w.\ \texttt{val-proc}\ \texttt{Mkt}\ p\ T\ w = \kappa\ w)\end{aligned}$$

$$\texttt{attainable} \quad :: \quad (\alpha \to \mathbb{R}) \to \mathbb{N} \to \mathbb{B}$$

$$\texttt{attainable}\ \kappa\ T \quad \Leftrightarrow \quad (\exists p.\ \texttt{replic-pf}\ p\ \kappa\ T)$$

$$\texttt{complete-market} \quad :: \quad (\alpha, \beta)\,\texttt{discrete-market} \to \mathbb{B}$$

$$\texttt{complete-market}\ \texttt{Mkt} \quad \Leftrightarrow \quad \forall T.\ \forall \kappa \in (\texttt{borel-measurable}\ \mathcal{F}_T).\ \texttt{attainable}\ \kappa\ T$$

lemma REPLICATING-FAIR-PRICE

assumes `viable-market Mkt`

and `replic-pf` $p\ \kappa\ T$

and `support-adapt Mkt` p

shows `fair-price Mkt (init-value` p`)` $\kappa\ T$

The Cox-Ross-Rubinstein setting

- Probability space: "infinite coin-toss space". Outcomes are sequences $\omega = \langle \omega_1, \ldots, \omega_n, \ldots \rangle$, where every ω_i is $\top$ with probability p and $\bot$ with probability $1 - p$

```
bernoulli-stream p  =
  stream-space (measure-pmf (bernoulli-pmf p))
```

- Filtration: information at time n is the outcome of the first n coin tosses

The Cox-Ross-Rubinstein setting

- Probability space: "infinite coin-toss space". Outcomes are sequences $\omega = \langle \omega_1, \ldots, \omega_n, \ldots \rangle$, where every ω_i is $\top$ with probability p and $\bot$ with probability $1 - p$

```
bernoulli-stream p  =
  stream-space (measure-pmf (bernoulli-pmf p))
```

- Filtration: information at time n is the outcome of the first n coin tosses
- Available assets:
  - Risk-free asset:initial value $R_0 = 1$ and

    $$R\ (n+1)\ \omega = (1+r).(R\ n\ \omega), \text{ with } r > -1$$

The Cox-Ross-Rubinstein setting

- Probability space: "infinite coin-toss space". Outcomes are sequences $\omega = \langle \omega_1, \ldots, \omega_n, \ldots \rangle$, where every ω_i is $\top$ with probability p and $\bot$ with probability $1 - p$

```
bernoulli-stream p  =
  stream-space (measure-pmf (bernoulli-pmf p))
```

- Filtration: information at time n is the outcome of the first n coin tosses
- Available assets:
  - Risk-free asset:initial value $R_0 = 1$ and

    $$R\ (n+1)\ \omega = (1+r).(R\ n\ \omega), \text{ with } r > -1$$

  - Risky asset: initial value S_0, $d < u$ and

    $$S\ (n+1)\ \omega\ = \begin{cases} u.(S\ n\ \omega) & \text{if } \omega_n \text{ is } \top \\ d.(S\ n\ \omega) & \text{if } \omega_n \text{ is } \bot \end{cases}$$

Main results

- In the CRR model, a market is viable if and only if $d < 1 + r < u$
- The market is complete in this model

CRR_Model.thy (%ISABELLE_HOME%\afp-2024-01-23\thys\DiscretePricing\)

```
definition (in disc_equity_market) complete_market where
"complete_market ⟷ (∀matur. ∀ der∈ borel_measurable (F matur).
  (∃p. replicating_portfolio p der matur))"

lemma (in CRR_market_viable) CRR_market_complete:
  shows "complete_market" unfolding complete_market_def
proof (intro allI impI)
  fix matur::nat
  show "∀ der ∈ borel_measurable (G matur).
    (∃p. replicating_portfolio p der matur)"
  proof
    fix der::"bool stream⇒real"
    assume "der ∈ borel_measurable (G matur)"
    define N where "N = bernoulli_stream ((1 + r - d) / (u - d))"
    hence "replicating_portfolio (delta_hedging N der matur) der matur"
      using delta_hedging_replicating
      ‹der ∈ borel_measurable (G matur)› by simp
    thus "∃pf. replicating_portfolio pf der matur" by auto
  qed
qed
```

Main results

- In the CRR model, a market is viable if and only if $d < 1 + r < u$
- The market is complete in this model

CRR_Model.thy (%ISABELLE_HOME%\afp-2024-01-23\thys\DiscretePricing\)

```
definition (in disc_equity_market) complete_market where
"complete_market ⟷ (∀matur. ∀ der∈ borel_measurable (F matur).
  (∃p. replicating_portfolio p der matur))"

lemma (in CRR_market_viable) CRR_market_complete:
  shows "complete_market" unfolding complete_market_def
proof (intro allI impI)
  fix matur::nat
  show "∀ der ∈ borel_measurable (G matur).
    (∃p. replicating_portfolio p der matur)"
  proof
    fix der::"bool stream⇒real"
    assume "der ∈ borel_measurable (G matur)"
    define N where "N = bernoulli_stream ((1 + r - d) / (u - d))"
    hence "replicating_portfolio (delta_hedging N der matur) der matur"
      using delta_hedging_replicating
      ‹der ∈ borel_measurable (G matur)› by simp
    thus "∃pf. replicating_portfolio pf der matur" by auto
  qed
qed
```

- The entire formalization is available on the *Archive of Formal Proofs*
  - `https://www.isa-afp.org/entries/DiscretePricing.html`
  - The entire formalization is over 350 pages long, even using Isabelle's automation

Other formalization efforts

On local hidden variables

Is quantum mechanics complete?

- Einstein did not think so
  - Disturbed by statistical nature of measurement outcomes
  - EPR paradox: *spooky action at a distance*

On local hidden variables

Is quantum mechanics complete?

- Einstein did not think so
  - Disturbed by statistical nature of measurement outcomes
  - EPR paradox: *spooky action at a distance*
- A suggested solution: hidden-variable models
  - *Local* hidden-variable models to account for relativity

On local hidden variables

Is quantum mechanics complete?

- Einstein did not think so
  - Disturbed by statistical nature of measurement outcomes
  - EPR paradox: *spooky action at a distance*
- A suggested solution: hidden-variable models
  - *Local* hidden-variable models to account for relativity
- The existence of such models is refuted by Bell's inequality (1964)
  - CHSH inequality: **Proposed experiment to test local hidden-variable theories** [Clauser, Horne, Shimony, Holt, 1969]
  - One of Bell's inequalities that was verified experimentally by A. Aspect

On local hidden variables

Is quantum mechanics complete?

- Einstein did not think so
  - Disturbed by statistical nature of measurement outcomes
  - EPR paradox: *spooky action at a distance*
- A suggested solution: hidden-variable models
  - *Local* hidden-variable models to account for relativity
- The existence of such models is refuted by Bell's inequality (1964)
  - CHSH inequality: **Proposed experiment to test local hidden-variable theories** [Clauser, Horne, Shimony, Holt, 1969]
  - One of Bell's inequalities that was verified experimentally by A. Aspect
- Formalization of this inequality and the accompanying Tsirelson bound
  - `https://www.isa-afp.org/entries/TsirelsonBound.html`
  - Total formalization (along with fundamental quantum notions) is over 250 pages long

The Superposition Calculus

- A saturation-based theorem proving technique in first-order logic

The Superposition Calculus

- A saturation-based theorem proving technique in first-order logic
- The most powerful technique for automated theorem proving in first-order logic with equality

The Superposition Calculus

- A saturation-based theorem proving technique in first-order logic
- The most powerful technique for automated theorem proving in first-order logic with equality
- All tools invoked by Sledgehammer implement some version of this calculus

The Superposition Calculus

- A saturation-based theorem proving technique in first-order logic
- The most powerful technique for automated theorem proving in first-order logic with equality
- All tools invoked by Sledgehammer implement some version of this calculus
- N. Peltier: formalization of the Superposition Calculus *within* Isabelle
    - `https://www.isa-afp.org/entries/SuperCalc.html`

Ongoing work on AMMs

- On Concentrated Liquidity Market Makers
  - Initial implementation with Uniswap v3
  - Other implementations on Solana (Orca CLMM, Raydium-Amm-v3)
  - Each protocol can implement its tick ranges, fees, etc

Ongoing work on AMMs

- On Concentrated Liquidity Market Makers
  - Initial implementation with Uniswap v3
  - Other implementations on Solana (Orca CLMM, Raydium-Amm-v3)
  - Each protocol can implement its tick ranges, fees, etc
- Question: can we compute an aggregated market depth and price slippage on several pools for a given pair of tokens?

Ongoing work on AMMs

- On Concentrated Liquidity Market Makers
  - Initial implementation with Uniswap v3
  - Other implementations on Solana (Orca CLMM, Raydium-Amm-v3)
  - Each protocol can implement its tick ranges, fees, etc
- Question: can we compute an aggregated market depth and price slippage on several pools for a given pair of tokens?
- Answer: yes, under some hypotheses on the prices in the pools
  - The proof outline is a couple of pages long
  - Goal: formalize the entire proof with Isabelle

Ongoing work on AMMs

- On Concentrated Liquidity Market Makers
  - Initial implementation with Uniswap v3
  - Other implementations on Solana (Orca CLMM, Raydium-Amm-v3)
  - Each protocol can implement its tick ranges, fees, etc
- Question: can we compute an aggregated market depth and price slippage on several pools for a given pair of tokens?
- Answer: yes, under some hypotheses on the prices in the pools
  - The proof outline is a couple of pages long
  - Goal: formalize the entire proof with Isabelle
- Main steps
  - Define an abstract representation of a pool by focusing on the properties of interest
  - Prove the result on this abstract representation
  - Prove that the concrete implementations are instances of the abstract representation

Ongoing work on AMMs

- On Concentrated Liquidity Market Makers
  - Initial implementation with Uniswap v3
  - Other implementations on Solana (Orca CLMM, Raydium-Amm-v3)
  - Each protocol can implement its tick ranges, fees, etc
- Question: can we compute an aggregated market depth and price slippage on several pools for a given pair of tokens?
- Answer: yes, under some hypotheses on the prices in the pools
  - The proof outline is a couple of pages long
  - Goal: formalize the entire proof with Isabelle
- Main steps
  - Define an abstract representation of a pool by focusing on the properties of interest
  - Prove the result on this abstract representation
  - Prove that the concrete implementations are instances of the abstract representation

Thank you for your attention